Quantum Computing: Systems and Architecture Project

2026MCS2232, 2026MCS2245, 2026MCS2261, 2026MCS2234
COL8422: Quantum Computing Systems and Architecture
Department of Computer Science and Engineering
Indian Institute of Technology Delhi
GitHub Repository: https://github.com/HarshV-910/QuantumProject.git

I. Introduction and Project Requirements

A pure n-qubit state contains 2^n complex amplitudes, but not every circuit requires an explicit statevector. Clifford circuits admit polynomial-time stabilizer simulation; states with modest entanglement can be compressed as MPS; and structured circuits can expose repeated subgraphs that a DD can share. The project therefore assigns four backends behind one Python-callable simulator interface, with C/C++ kernels and a common correctness and benchmarking harness.

The required interface includes initialization, supported one- and two-qubit gates, measurement, sampling, and Pauli-string expectation values. A backend must raise an unsupported-operation error for gates outside its representation's supported set; silent fallback to another simulator is prohibited. The specification's correctness gate requires fidelity $F \geq 0.999$ for applicable circuits through 20 qubits. The primary performance metric is the maximum n reached within 120 seconds and 48 GB RAM on the backend's assigned workload. The final report is required to be 6–8 pages in a two-column conference style.

II. Architecture and Representation Trade-offs

A. Member A: Statevector (universal circuits)

The statevector backend stores $|\psi\rangle = \sum_{x=0}^{2^n-1} \alpha_x |x\rangle$ explicitly. Its raw complex128 state requires approximately $16 \cdot 2^n$ bytes, and a gate generally costs $O(2^n)$ work. The Python adapter validates calls and delegates state operations to native code, with a CUDA execution path. Supported operations reported by the member include H, X, Y, Z, S, T, U, CNOT, CZ, SWAP, CU, measurement, sampling, and Pauli expectations. This is the most general representation among the four, but both memory and per-gate work scale exponentially.

B. Member B: Stabilizer tableau (Clifford circuits)

The stabilizer backend implements the Aaronson–Gottesman tableau with n stabilizer and n destabilizer rows plus phase information. Each row stores x and z bits per qubit; the C++ kernel packs 64 qubits per machine word and is exposed through a Python interface using `ctypes`. Storage is approximately $4n^2$ bits, Clifford gate updates cost $O(n)$, and measurement costs $O(n^2)$. The backend rejects non-Clifford operations such as general $T/U/CU$ rather than silently changing representations. Sampling is implemented from the affine subspace of computational-basis outcomes, and Pauli expectations are exact $(-1, 0, +1)$.

C. Member C: Matrix product state

The MPS backend represents a state as a chain of site tensors,

$$|\psi\rangle = \sum_{i_1,\ldots,i_n} A^{[1]i_1} \cdots A^{[n]i_n} |i_1 \cdots i_n\rangle, \qquad (1)$$

where each tensor has dimensions $\chi_{k-1} \times 2 \times \chi_k$. For approximately uniform bond dimension χ, storage is $O(n\chi^2)$ rather than $O(2^n)$. Single-site gates update one tensor; two-site updates use an SVD, with typical cost $O(\chi^3)$ for comparable bounded local dimensions. Non-adjacent gates are routed using SWAPs. The implementation uses NumPy for tensor operations/SVD and C++ kernels for selected tensor updates, exposed with `ctypes`. Its advantage depends on entanglement and bond dimensions; arbitrary states can still require exponential resources.

D. Member D: Weighted decision diagram

The DD backend represents a state as a directed graph with complex edge weights and low/high branches at qubit levels. Canonical edge normalization and a unique table merge equivalent subgraphs. A C++ core performs

TABLE I
REPRESENTATION-LEVEL TRADE-OFFS (ASYMPTOTIC, WORKLOAD-DEPENDENT).

Backend	Main advantage	Main limitation
Statevector	Universal gate support; direct amplitudes	$O(2^n)$ memory and per-gate work
Tableau	$O(n^2)$ storage and polynomial Clifford simulation	Does not directly support general non-Clifford gates
MPS	Compact for low-entanglement states	Cost rises with bond dimension; SWAP/SVD overhead
DD	Shares repeated subgraphs in structured circuits	Graph size depends strongly on circuit structure

recursive gate operations and is wrapped by Python. The representation can reduce storage and work when many subgraphs are reusable, but compactness is circuit-dependent and is not guaranteed for random or highly entangling workloads. The reported interface includes one- and two-qubit gates, measurement, sampling, amplitudes, and Pauli expectations.

III. CORRECTNESS EVIDENCE

Table II summarizes the test evidence in the supplied member reports. These are member-specific local results; they do not, by themselves, prove that the integrated group interface passes on the course evaluation machine.

For statevector, GHZ and QFT fidelity was reported as 1.000000 at all tested sizes; BV's expected measurement mode appeared in 100 shots per size. A separate seeded random-universal comparison recorded fidelity 1.0 at $n = 4$ and successful checks against the $F \geq 0.999$ threshold at $n = 8, 12, 16$; exact numeric values for the latter sizes were not retained. The stabilizer report states bit-identical agreement between C++ and NumPy kernels across word boundaries at $n = 63, 64, 65, 130$. It correctly skips QFT in the stabilizer harness because controlled-phase gates are not Clifford. MPS reports fidelity 1.0 for GHZ and QFT, with BV's expected bitstring mode observed in 100 shots per test. A group-level fidelity matrix covering all required circuit classes and all backends remains necessary.

TABLE II
CORRECTNESS EVIDENCE REPORTED BY INDIVIDUAL BACKEND OWNERS.

Backend	Test evidence supplied	Result
Statevector	GHZ, QFT, BV at $n = 4, 8, 12, 16, 20$	15/15 passed
Stabilizer	Course harness: GHZ/BV for $n \leq 20$; unit tests cover random Clifford circuits, measurements, expectations, sampling, and error handling	10/10 harness; 9/9 unit tests
MPS	GHZ, QFT, BV at $n = 4, 8, 12, 16, 20$; rebuilt native library and reran	15/15 passed
DD	Member report describes benchmark harness/CSV measurements; no consolidated numeric fidelity table was supplied	Not established by supplied report

IV. INTERFACE SEMANTICS AND INTEGRATION CONSIDERATIONS

The common interface deliberately separates circuit semantics from representation internals. Initialization must produce $|0\rangle^{\otimes n}$; gate calls must apply the specified operator to the selected qubits; measurement must return a bit and collapse the represented state; sampling must return a measurement distribution; and expectation must evaluate a Pauli string in the current state. This common contract makes backend comparisons meaningful only when the same circuit, parameters, initial state, and timing boundary are used.

The four representations have different legal operation sets. Statevector supports the general gate set and is the reference representation for arbitrary circuits. Stabilizer supports Clifford operations and must reject unsupported non-Clifford gates. MPS and DD have their own implementation-supported gates, and must expose errors consistently with the common interface. A correct integration layer should not silently route an unsupported call to another backend: doing so changes the algorithm being benchmarked and conceals unsupported functionality. The member reports describe individual Python adapters and native kernels, but the supplied evidence does not include a single end-to-end integration-harness transcript for all four backends.

Several implementation choices also affect observed timings. Python-to-native call overhead can dominate small circuits; memory allocation and state initialization

TABLE III
EXPECTED SCALING AND BENCHMARK IMPLICATION.

Backend	Basic cost model	Prediction for assigned workload
Statevector	$O(2^n)$ memory; $O(G2^n)$ work	Universal simulation; GPU may improve constants, not exponent
Tableau	$O(n^2)$ bits; $O(n)$ per Clifford gate	Should dominate dense statevector on Clifford circuits
MPS	$O(n\chi^2)$ memory; two-site SVD often $O(\chi^3)$	Should work well while QAOA entanglement keeps χ modest
DD	Graph-size dependent; repeated subgraphs can be shared	Structured/oracle circuits should benefit; random circuits may expand

affect statevector timings; SVD and SWAP routing affect MPS; tableau row operations and sampling affect stabilizer; and node creation, normalization, unique-table lookups, and graph traversal affect DD. Therefore, reported wall-clock times must be interpreted alongside each report's timing boundary. They are not interchangeable kernel-only measurements.

V. COMPLEXITY PREDICTIONS AND WHAT THE RESULTS TEST

Table III summarizes the expected trends before considering measured results. These predictions are representation-level models, not guarantees for every circuit instance.

The supplied results broadly align with these predictions. The tableau is much faster than Aer on the tested Clifford family, and the DD backend is fast on GHZ, QFT, and BV relative to its own more demanding workload results. The statevector comparison shows a growing advantage in the recorded local run as the circuit grows, though the exact crossover cannot be inferred beyond the tested range. The MPS report shows low runtimes for its local QAOA sweep, but the absence of the plot and intermediate values in the supplied artifacts prevents a complete independent inspection of the curve. These observations should be treated as evidence for workload specialization, not as a single cross-backend leaderboard.

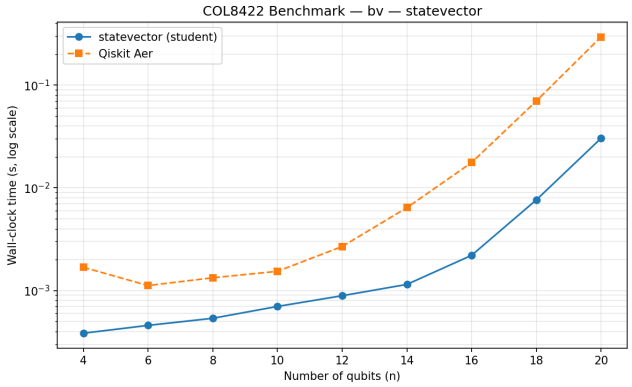


Fig. 1. Preliminary statevector/Aer runtime scaling for random-universal circuits.

TABLE IV
SELECTED STATEVECTOR TIMINGS FROM THE SUPPLIED LOCAL RUN.

n	Student (s)	Aer (s)	Aer/student
4	0.005099	0.005614	1.10×
10	0.028008	0.031544	1.13×
14	0.040073	0.141774	3.54×
18	0.168901	1.735397	10.27×
20	0.539250	8.484346	15.73×

VI. PERFORMANCE RESULTS

A. Statevector: random-universal circuits

The supplied statevector measurements are preliminary Colab timings for seeded random-universal circuits with a depth factor of $5n$, random $U(\theta, \phi, \lambda)$ gates, and alternating brick-wall CNOT layers. Three timed repetitions were summarized by their median after an initial Aer execution. Timings include simulator/circuit setup and Python/native overhead, rather than isolating the CUDA kernel. The student implementation was faster at all nine reported sizes from $n = 4$ to 20. The measured 15.73× ratio at $n = 20$ is promising, but the sweep does not determine the maximum reachable n under the prescribed 120-second/48-GB limit and is not an official result. Both statevector implementations have $O(2^n)$ storage and approximately $O(G2^n)$ gate-update work; any advantage is a constant-factor engineering improvement, not an asymptotic one.

B. Stabilizer: random Clifford circuits

The stabilizer member benchmarked the course's near-Clifford generator with $k = 0$ T gates, i.e. a Clifford

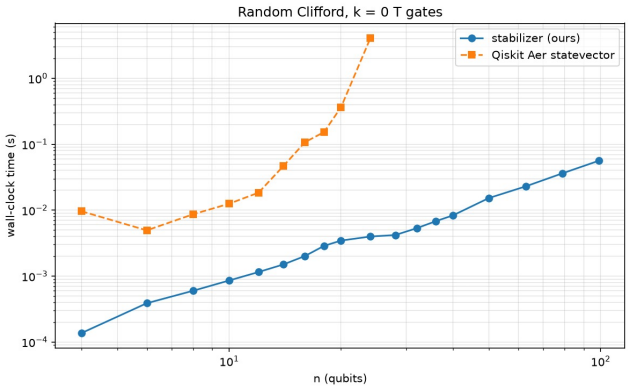


Fig. 2. Member B's reported random-Clifford runtime plot.

TABLE V
SELECTED STABILIZER RESULTS FROM THE SUPPLIED REPORT.

n	Gates	Tableau (s)	Aer (s)
4	100	1.4×10^{-4}	9.7×10^{-3}
12	624	1.2×10^{-3}	1.8×10^{-2}
20	1,680	3.4×10^{-3}	0.36
24	2,400	4.0×10^{-3}	4.0
99	39,600	5.6×10^{-2}	–

workload, and compared initialization plus gate application with Qiskit Aer statevector. The supplied report gives median-of-three timings. The report states the tableau is faster at every tested size, with approximately $100\times$ speedup at $n = 20$ and $1000\times$ at $n = 24$. Aer data are absent at $n = 99$. The reported extrapolation to a maximum size is a prediction, not a measured resource-limit result. For non-Clifford $k > 0$ workloads, the backend as supplied rejects unsupported gates; it therefore does not provide a valid runtime comparison for those benchmark cases.

C. MPS: shallow one-dimensional QAOA

The MPS report supplies a local QAOA run from $n = 4$ to $n = 40$, with MPS runtime increasing from approximately 0.0006 s to 0.0059 s. The referenced plot file was not included among the supplied artifacts, and the report does not provide intermediate numeric points; a full reproducible curve cannot be reconstructed without inventing measurements. It also states that QAOA parameters were generated independently for MPS and Aer, so the comparison is not controlled on identical circuits. The report describes substantial variability in Aer timings and some Aer memory failures. Accordingly, the supplied measurements support only a preliminary

TABLE VI
MPS QAOA DATA ACTUALLY PRESENT IN THE SUPPLIED REPORT.

Qubit count	MPS runtime
4	≈ 0.0006 s
40	≈ 0.0059 s

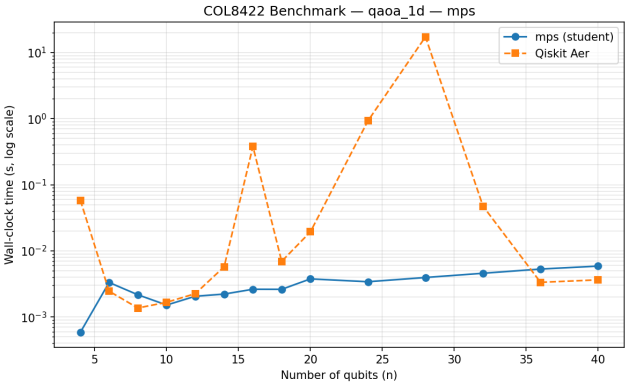


Fig. 3. Local QAOA runtime comparison; vertical axis is logarithmic.

local MPS scaling observation, not a definitive speedup claim.

For low-entanglement states, MPS storage is $O(n\chi^2)$ and local two-site updates are typically cubic in bond dimension. This predicts an advantage when bond dimensions remain modest, but entanglement growth can eliminate the benefit. A controlled evaluation should sweep the assigned shallow-QAOA sizes and depths with identical parameters for both MPS and Aer.

D. Decision diagrams: structured and demanding workloads

The DD report includes six workload plots and endpoint measurements. Figures **??** and **??** reproduce the supplied plots; the source report states that runtime is the median of three runs per size and its prescribed peak-memory metric is recorded on the first run. Aer data were supplied for GHZ, QFT, BV, and QAOA, but not near-Clifford or random-universal. Comparisons should be interpreted cautiously because the reports warn that random circuit instances or parameters may differ between backends.

The structured GHZ/QFT/BV cases remain in the millisecond range at 20 qubits. Costs rise for less-shareable workloads: near-Clifford reaches approximately 2.88 s at 20 qubits and random-universal approximately 54.01 s at 14 qubits. These observations are consistent with DD performance depending on equivalent subgraph

TABLE VII
DD ENDPOINT RUNTIMES FROM THE SUPPLIED CSV SUMMARY.

Circuit	Largest n	DD median (s)
GHZ	20	0.00469
QFT	20	0.01119
BV	20	0.00569
1D QAOA	14	2.29166
Near-Clifford	20	2.87772
Random-universal	14	54.01464

sharing; they do not establish a universal complexity bound. The DD report does not include a consolidated numeric fidelity table, so its benchmark times should not be interpreted as proof that all correctness requirements were met.

VII. BENCHMARK PROTOCOL AND REPRODUCIBILITY

For graded results, the specification names an Ubuntu 22.04 environment with GCC 12, CUDA 11.5, Python 3.10, and NVIDIA A40 GPUs; the per-group budget is 48 GB RAM and 16 CPU cores. The required maximum-size metric is the largest qubit count reachable within 120 seconds and the memory budget while passing correctness checks. The report must plot runtime against n for the assigned workload and overlay Qiskit Aer statevector (fp64), then report the crossover if observed. If a crossover is not observed, the report should explain quantitatively why the representation's complexity predicts that outcome.

The reproducible sequence for the final group run should include a clean build with a plain `make`, an individual correctness harness run for each backend, the integration harness, and the benchmark sweeps on the designated evaluation machine. For randomized circuits, the same circuit instances and parameters should be passed to both student and Aer implementations; otherwise, differences in circuit difficulty confound the runtime comparison. Report the timing statistic (the supplied member reports generally use the median of three runs), initialization and compilation policy, memory measurement procedure, random seed, circuit depth, and any failed or skipped cases. Save raw CSVs and plot files alongside the report.

The supplied evidence has differing coverage. Statevector reports a nine-point random-universal sweep through 20 qubits; stabilizer reports Clifford measurements through 99 qubits but no Aer value at 99; MPS reports QAOA endpoint values through 40 qubits without the referenced plot asset; DD reports six circuit-family plots and endpoints through 14 or 20 qubits. None of these alone documents the required maximum reachable size under the official 120-second/48-GB constraints for every assigned class. Consequently, the report avoids presenting an extrapolation as a measured maximum.

VIII. CROSS-BACKEND ANALYSIS AND LIMITATIONS

The results support the central design premise: representation choice matters. The statevector is universal but exponential; the tableau is highly effective for Clifford circuits; MPS can compress low-entanglement states; and DDs can exploit repeated structure. The reported best results are workload-specific and were obtained in different local environments, so absolute times should not be ranked across backends. Only within-report comparisons with sufficiently similar setups are meaningful.

The supplied material does not establish every project deliverable. In particular: (i) official A40/CUDA 11.5 benchmark runs are not supplied; (ii) the statevector sweep ends at $n = 20$, the DD sweep endpoints vary by circuit, and the MPS QAOA report provides only endpoint numbers in the available source; (iii) no measured maximum n under 120 seconds and 48 GB RAM is available for all assigned classes; (iv) the stabilizer implementation does not support the required non-Clifford $k = 5, 10, 20$ benchmark cases; (v) no group-level integration-harness result or consolidated correctness/fidelity matrix was supplied; and (vi) the MPS plot file referenced in its member report was not included. These are evidence gaps, not assumptions that the code necessarily fails.

IX. CONCLUSION

The four implementations demonstrate complementary simulation representations and report encouraging member-level correctness and performance results on their natural workloads. The statevector backend shows a preliminary 15.73× advantage over Aer at 20 qubits in the supplied Colab run; the stabilizer tableau has large reported gains on Clifford circuits; MPS reports low local QAOA runtimes through 40 qubits; and DDs handle structured circuits efficiently while showing sharply higher costs on less-shareable workloads. To satisfy the specification conclusively, the group must run the final harness and assigned scaling sweeps on the official evaluation environment, record the measured maximum n under the prescribed resource limits, compare identical circuits against Aer, and attach the group integration and correctness evidence. No preliminary local result in this report is represented as an official graded measurement.

REFERENCES

[1] COL8422, "Building a Simulator for Gate Based Quantum Computing: Part 1 – Multi-Backend Simulator," course project specification, 2026.

[2] T. Häner and D. Steiger, "0.5 Petabyte Simulation of a 45-Qubit Quantum Circuit," in *Proc. SC*, 2017.

[3] S. Aaronson and D. Gottesman, "Improved Simulation of Stabilizer Circuits," *Physical Review A*, vol. 70, 052328, 2004.

[4] G. Vidal, "Efficient Classical Simulation of Slightly Entangled Quantum Computations," *Physical Review Letters*, vol. 91, 147902, 2003.

[5] A. Viamontes, I. Markov, and J. Hayes, "Improving Gate-Level Simulation of Quantum Circuits," 2003.

[6] Qiskit Aer, "AerSimulator: statevector method," https://github.com/Qiskit/qiskit-aer.